第 13 章　迁移学习

实际应用中很多任务的数据的标注成本很高，无法获得充足的训练数据，这种情况可以使用**迁移学习（transfer learning）**。假设 A、B 是两个相关的任务，A 任务有很多训练数据，就可以把从 A 任务中学习到的某些可以泛化知识迁移到 B 任务。迁移学习有很多分类，本章介绍了**领域自适应（domain adaptation）**和**领域泛化（domain generalization）**。

13.1　领域偏移

目前为止，我们学习了很多深度学习的模型，所以训练一个分类器比较简单，比如要训练数字的分类器，给定训练数据，训练好一个模型，应用在测试数据就结束了。

如图 13.1 所示，数字识别这种简单的问题，在基准数据集 MNIST [1] 上能做到 99.91% 的正确率[2]。但测试数据和训练数据的分布不一样时会导致一些问题。假设训练时数字是黑白的，但测试时数字是彩色的。常见的误区是：虽然数字的颜色不同，但对于模型，其形状是一样的。如果模型能识别出黑白的图片中的数字，其应该也能识别出彩色图片的数字。但实际上，如果使用黑白的数字图像训练一个模型，直接用到彩色的数字上，正确率会非常低。MNIST 数据集中的数字是黑白的，MNIST-M 数据集[3] 中的数字是彩色的，如果在 MNIST 数据集上训练，在 MNIST-M 数据集上测试，正确率只有 52.25%[4]。一旦训练数据跟测试数据分布不同，在训练数据上训练出来的模型，在测试数据上面可能效果不好，这种问题称为领域偏移（domain shift）。

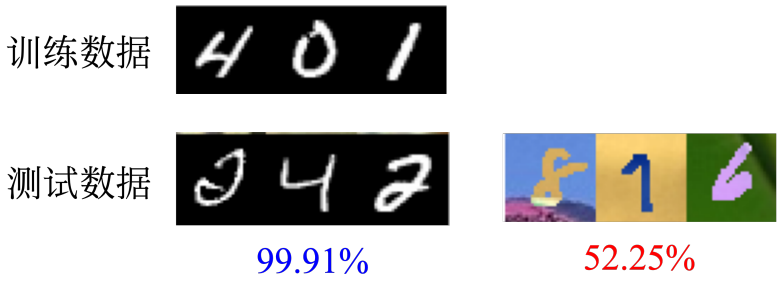


图 13.1　数据分布不同导致的问题

领域偏移其实有很多种不同的类型，模型输入的数据分布有变化的状况是一种类型。另外一种类型是，输出的分布也可能有变化。比如在训练数据上面，可能每一个数字出现的概率都是一样的，但是在测试数据上面，可能每一个数字输出的概率是不一样的，有可能某一个数字它输出的概率特别大，这也是有可能的。还有一种比较罕见状况的类型是，输入跟输出虽然分布可能是一样的，但它们之间的关系变了。比如同一张图片在训练数据里的标签为“0”，但是在测试数据的标签为“1”。

接下来我们专注于输入数据不同的领域偏移。如图 13.2 所示，接下来我们称测试数据来自目标领域（target domain），训练数据来自源领域（source domain），因此源领域是训练数据，目标领域是测试数据。

在基准数据集上学习时，很多时候无视领域偏移问题，假设训练数据跟测试数据往往有一样的分布，在很多任务上面都有极高的正确率。但在实际应用时，当训练数据跟测试数据有一点差异时，机器的表现可能会比较差，因此需要领域自适应来提升机器的性能。对于领域自

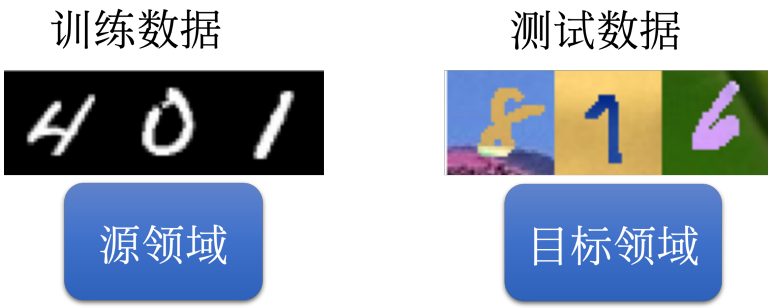


图 13.2　源领域与目标领域

适应，训练数据是一个领域，测试数据是另外一个领域，要把某一个领域上学到的信息用到另外一个领域，领域自适应侧重于解决特征空间与类别空间一致，但特征分布不一致的问题。

13.2　领域自适应

接下来介绍下领域自适应，以手写数字识别为例。比如有一堆有标注的训练数据，这些数据来自源领域，用这些数据训练出一个模型，这个模型可以用在不一样的领域。在训练的时候，我们必须要对测试数据所在的目标领域有一些了解。

随着了解的程度不同，领域自适应的方法也不同。如果目标领域上有一大堆有标签的数据，这种情况其实不需要做领域自适应，直接用目标领域的数据训练。如果目标领域上有一点有标签的数据，这种情况可以用领域自适应，可以用这些有标注的数据微调在源领域上训练出来的模型。这边的微调跟 BERT 的微调很像，已经有一个在源领域上训练好的模型，只要拿目标领域的数据跑个两、三个回合就足够了。在这一种情况下，需要注意的问题是，因为目标领域的数据量非常少，所以要小心不要过拟合，不要在目标领域的数据上迭代太多次。在目标数据上迭代太多次，可能会过拟合到目标领域的少量数据上，在真正的测试集的表现可能就不好。

> 为了避免过拟合的情况，有很多的解决方法，比如调小一点学习率。要让微调前、后的模型的参数不要差很多，或者让微调前、后的模型的输入跟输出的关系，不要差很多等等。

下面主要介绍下在目标领域上有大量未标注的数据的这种情况。这种情况其实是很符合实际会发生的情况。比如在实验室里面训练了一个模型，并想要把它用在真实的场景里面，于是将模型上线。上线后的模型确实有一些人来用，但得到的反馈很差，大家嫌弃系统正确率很低。这种情况就可以用领域自适应的技术，因为系统已经上线后会有人使用，就可以收集到一大堆未标注的数据。这些未标注的数据可以用在源领域上训练一个模型，并用在目标领域。

最基本的想法如图 13.3 所示，训练一个特征提取器（feature extractor）。特征提取器也是一个网络，这个网络输入是一张图片，输出是一个特征向量。虽然源领域与目标领域的图像不一样，但是特征提取器会把它们不一样的部分去除，只提取出它们共同的部分。虽然源领域和目标领域的图片的颜色不同，但特征提取器可以学习到把颜色的信息滤掉，忽略颜色。源领域和目标领域的图片通过特征提取器以后，其得到的特征是没有差异的，分布相同。通过特

征提取器可以在源领域上训练一个模型，直接用在目标领域上。通过领域对抗训练（domain adversarial training）可以得到领域无关的表示。

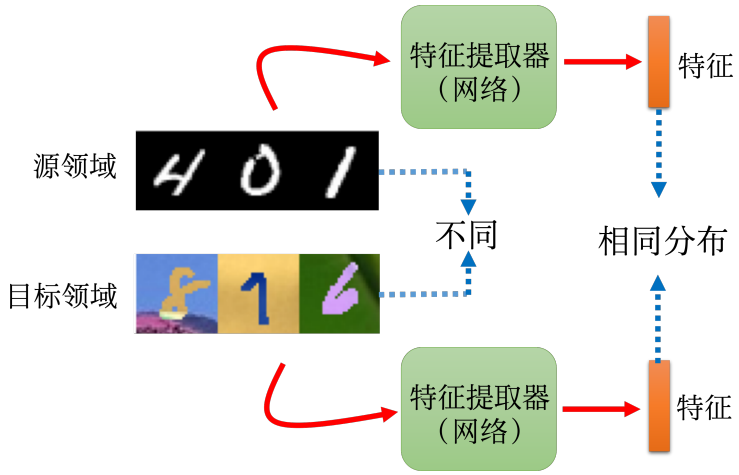


图 13.3　通过特征提取器过滤颜色信息

一般的分类器可分成特征提取器和标签预测器（label predictor）两个部分。图像的分类器输入一张图像，输出分类的结果。假设图像的分类器有 10 层，前 5 层是特征提取器，后 5 层是标签预测器。前 5 层可看成特征提取器，一个图像通过前 5 层，其输出是一个向量；如果使用卷积神经网络，其输出是特征映射，但特征映射“拉直”也可以看做是一个向量，该向量再输入到后面 5 层（标签预测器）来产生类别。

> Q：为什么分类器的前 5 层是特征提取器，而不是前 1/2/3/4 层?
> A：分类器里面哪些部分算特征提取器，哪些部分算标签预测器，这个是由自己决定的，可以自行调整。

图 13.4 给出了特征提取器和标签预测器的训练过程。对于源领域上标注的数据，把源领域的数据“丢”进去，这跟训练一个一般的分类器一样，它通过特征提取器，再通过标签预测器，可以产生正确的答案。但不一样的地方是，目标领域的一堆数据是没有任何标注的，把这些图片“丢”到图像分类器，把特征提取器的输出拿出来看，希望源领域的图片“丢”进去的特征跟目标领域的图片“丢”进去的特征相同。图 13.4 中蓝色的点表示源领域图片的特征，红色的点表示目标领域图片的特征，通过领域对抗训练让蓝色的点跟红色的点分不出差异。

如图 13.5 所示，我们要训练一个领域分类器。领域分类器是一个二元的分类器，其输入是特征提取器输出的向量，其目标是判断这个向量是来自于源领域还是目标领域，而特征提取器学习的目标是要去想办法骗过领域分类器。领域对抗训练非常像是生成对抗网络，特征提取器可看成生成器，领域分类器可看成判别器。但在领域对抗训练里面，特征提取器优势太大了，其要骗过领域分类器很容易。比如特征提取器可以忽略输入，永远都输出一个零向量。这样做领域分类器的输入都是零向量，其也无法判断该向量的领域。但标签预测器也需要特征判断输入的图片的类别，如果特征提取器只会输出零向量，标签预测器无法判断是哪一张图片。特征提取器还是需要产生向量来让标签预测器可以输出正确的预测。因此特征提取器不能永远都输出零向量。

假设标签预测器的参数为 $\boldsymbol{\theta}_p$，领域分类器的参数为 $\boldsymbol{\theta}_d$，特征提取器的参数为 $\boldsymbol{\theta}_f$。源领

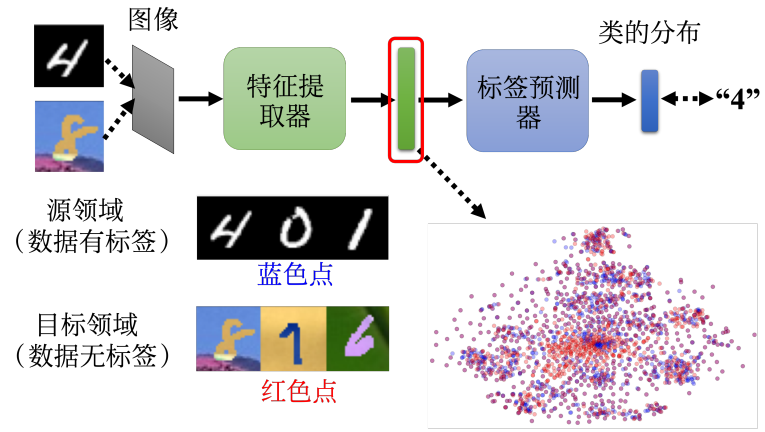


图 13.4　训练特征提取器，让源领域和目标领域的特征无差异[4]

域的图像是有标签的，所以可以计算它们的交叉熵来得出损失 L。领域分类器要想办法判断图片是源领域还是目标领域，这就是一个二元分类的问题，该分类问题的损失为 L_d。我们要去找一个 $\boldsymbol{\theta}_p$，它可以让 L 越小越好，即

$$\boldsymbol{\theta}_p^* = \min_{\boldsymbol{\theta}_p} L \tag{13.1}$$

我们要去找一个 $\boldsymbol{\theta}_d$，它可以让这个 L_d 越小越好，即

$$\boldsymbol{\theta}_d^* = \min_{\boldsymbol{\theta}_d} L_d \tag{13.2}$$

标签预测器要让源领域的图像分类越正确越好，领域分类器要让领域的分类越正确越好。而特征提取器站在标签预测器这边，它要去做领域分类器相反的事情，所以特征提取器的损失是标签预测器的损失 L 减掉领域分类器的损失 L_d，所以特征提取器的损失是 $L - L_d$，找一组参数 $\boldsymbol{\theta}_f$ 让 $L - L_d$ 的值越小越好，即

$$\boldsymbol{\theta}_f^* = \min_{\boldsymbol{\theta}_f} L - L_d \tag{13.3}$$

假设领域分类器的工作是把源领域跟目标领域分开，根据特征提取器的特征，来判断数据是来自源领域还是目标领域，把源领域和目标领域的两组特征分开。而特征提取器的损失中是 $-L_d$，这意味着它要做的事情跟领域分类器相反。如果领域分类器根据某张图片的特征判断这张图片属于源领域，而特征提取器要让领域分类器根据这张图片的特征判断这张图片属于目标领域，这样做也就可以分开源领域和目标领域的特征。本来领域分类器是让 L_d 的值越小越好，特征提取器要让 L_d 的值越大越好，其目的都是分开源领域跟目标领域的特征。以上是最原始的领域对抗训练做法。

领域对抗训练最原始的论文做了如图 13.6 所示的四个从源领域到目标领域的任务。如果用目标领域的图片训练，目标领域的图片测试，结果如表 13.1 所示，每一个任务正确率都是 90% 以上。但如果用源领域训练，目标领域测试，结果比较差。如果使用领域对抗训练，正确率会有明显的提升。

领域对抗训练最早的论文发表在 2015 年的 ICML 上面，其比生成对抗网络还要稍微晚一点，不过它们几乎可以是同时期的作品。

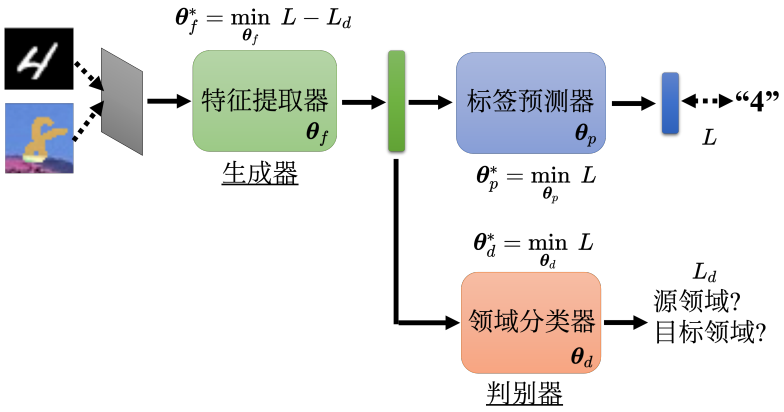


图 13.5　领域对抗训练



图 13.6　领域对抗训练最原始论文使用的任务

刚才这整套想法，有一个小小的问题。用蓝色的圆圈和三角形表示源领域上的两个类别，用正方形来表示目标领域上无类别标签的数据。可以找一个边界去把源领域上的两个类别分开。训练的目标是要让正方形的分布跟圆圈、三角形合起来的分布越接近越好。在图 13.7（a）所示的情况中，红色的点跟蓝色的点是挺对齐在一起的。在图 13.7 （b）所示的情况中，红色的点跟蓝色的点是分布挺接近的。虽然正方形的类别是未知的，但蓝色的圆圈跟蓝色的三角形的决策边界是已知的，应该让正方形远离决策边界。因此两种情况相比，我们更希望在图 13.7 （b）的情况发生，而避免让在图 13.7 （a）的状况发生。

让正方形远离边界（boundary）最简单的做法如图 13.8 所示。把很多无标注的图片先“丢”到特征提取器，再“丢”到标签预测器，如果输出的结果集中在某个类别上，就是离边界远；如果输出的结果每一个类别非常地接近，就是离边界近。除了上述比较的简单的方法外，还可以使用 DIRT-T[5]、最大分类器差异（maximum classifier discrepancy）[6] 等方法。这些方法在领域自适应中是不可或缺的。

目前为止都假设源领域跟目标领域的类别都要是一模一样，比如图像分类，源领域有老虎、狮子跟狗，目标领域也应该要有老虎、狮子跟狗，但实际上目标领域是没有标签的，其里

表 13.1　不同源领域和目标领域的数字图像分类的准确率

方法	源领域	MNIST	合成数字	SVHN	合成标志
	目标领域	MNIST-M	SVHN	MNIST	GTSRB
只使用源领域数据训练		57.49%	86.65%	59.19%	74.00%
使用领域对抗训练		81.49%	90.48%	71.07%	88.66%
只使用目标领域数据训练		98.91%	92.44%	99.51%	99.87%

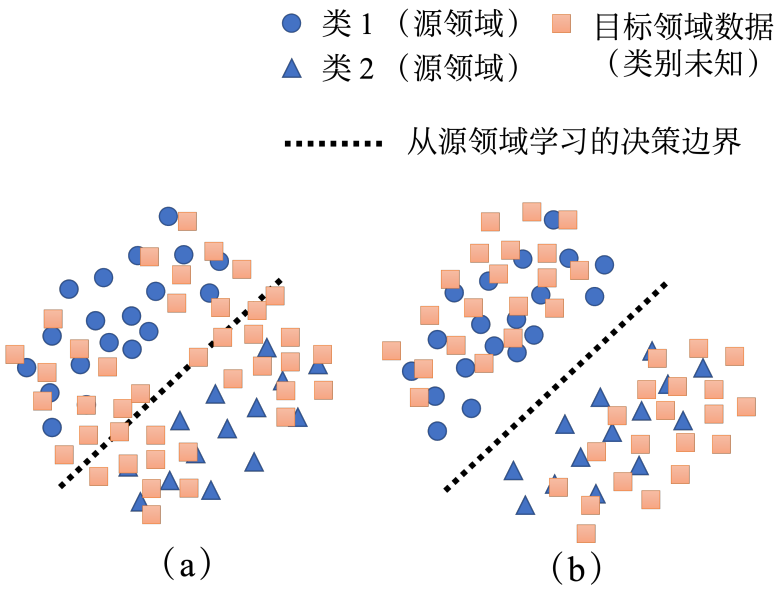


图 13.7　决策边界

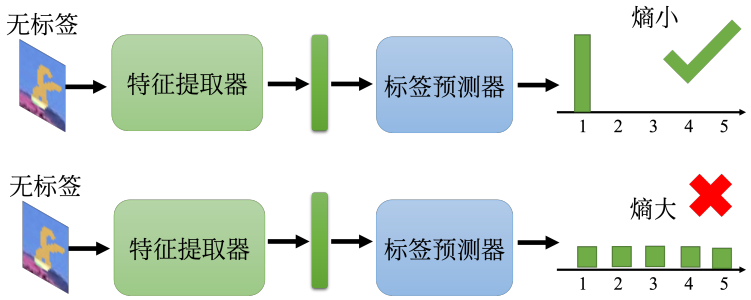


图 13.8　离边界越远越好

面的类别是未知的。如图 13.9 所示，实线的椭圆圈代表源领域里面有的东西，虚线的椭圆圈代表目标领域里面有的东西。图 13.9（a）中源领域里面的东西比较多，目标领域里面的东西比较少；图 13.9（b）中源领域里面的东西比较少，目标领域的东西比较多。图 13.9（c）中两者虽然有交集，但是各自都有独特的类别。

强制把源领域跟目标领域完全对齐在一起是有问题的，比如图 13.9（c）里面，要让源领域的数据跟目标领域的数据的特征完全匹配，这意味是要让老虎去变得跟狗像，或者让老虎变得跟狮子像，这样老虎这个类别就不能区分了。源领域跟目标领域有不同标签问题的解决方法，可参考论文“Universal Domain Adaptation”[7]。

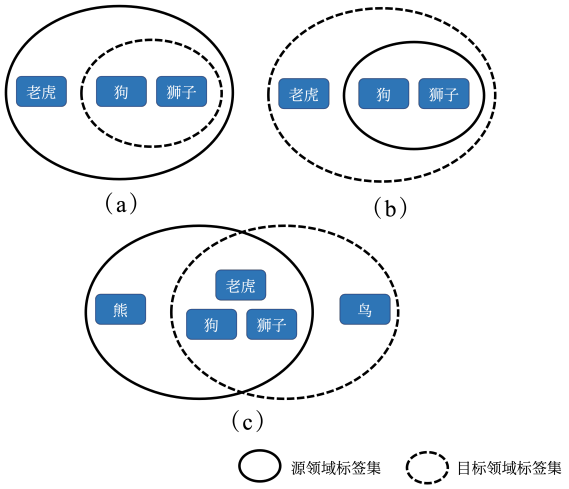


图 13.9　强制完全对齐源领域跟目标领域的问题

Q：如果特征提取器是卷积神经网络，而不是线性层（linear layer）。领域分类器输入是特征映射，特征映射本来就有空间的关系。把两个领域“拉”在一起会不会有影响**隐空间（latent space）**，让隐空间没能学到本来希望它学到的东西？
A：会有影响。领域自适应训练需要同时做好两个方面的事：一方面要骗领域分类器，另一方面是要让分类变正确。即不仅要把两个领域对齐在一起，还要让隐空间的分布是正确的。比如我们觉得 1 跟 7 比较像，为了要让分类器做好，特征提取器会让 1 跟 7 比较像。因为要提高标签预测器的性能，所以隐表示（latent representation）里面的空间仍然是一个比较好的隐空间。但如果给领域分类器就是要骗过领域分类器，这件事情的权重太大。模型就会学到只想骗过领域分类器，它就不会产生好的隐空间。

但是有一个可能是目标领域的数据不仅没有标签，而且还很少，比如目标领域只有一张图片，也就无法跟源领域对齐。这种情况可使用测试时训练（Testing Time Training，TTT）方法，读者可参考论文“Test-Time Training with Self-Supervision for Generalization under Distribution Shifts”[8]。

13.3　领域泛化

对目标领域一无所知，并不是要适应到某一个特定的领域上的问题通常称为领域泛化。领域泛化可又分成两种情况。一种情况是训练数据非常丰富，包含了各种不同的领域，测试数据只有一个领域。如图 13.10（a）所示，比如要做猫狗的分类器，训练数据里面有真实的猫跟狗的照片、素描的猫跟狗的照片、水彩画的猫跟狗的照片，期待因为训练数据有多个领域，模型可以学到如何弥平领域间的差异。当测试数据是卡通的猫跟狗时，模型也可以处理，具体细节可参考论文“Domain Generalization with Adversarial Feature Learning”[9] 。另外一种情况如图 13.10（b）所示，训练数据只有一个领域，而测试数据有多种不同的领域。虽然只有一个领域的数据，但可以想个数据增强的方法去产生多个领域的数据，具体可参考论文“Learning to Learn Single Domain Generalization”[10]。

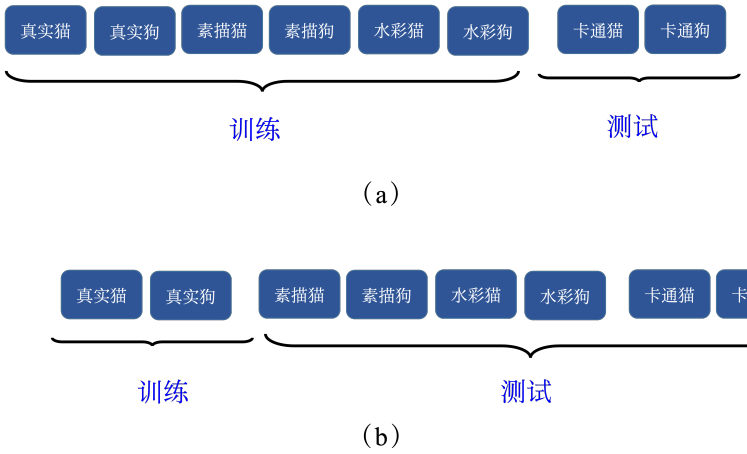


图 13.10　领域泛化示例

参考文献

[1] LECUN Y, BOTTOU L, BENGIO Y, et al. Gradient-based learning applied to document recognition[J]. Proceedings of the IEEE, 1998, 86(11): 2278-2324.

[2] AN S, LEE M, PARK S, et al. An ensemble of simple convolutional neural network models for mnist digit recognition[J]. arXiv preprint arXiv:2008.10400, 2020.

[3] GANIN Y, USTINOVA E, AJAKAN H, et al. Domain-adversarial training of neural networks[J]. The journal of machine learning research, 2016, 17(1): 2096-2030.

[4] GANIN Y, LEMPITSKY V. Unsupervised domain adaptation by backpropagation[C]// International conference on machine learning. PMLR, 2015: 1180-1189.

[5] SHU R, BUI H H, NARUI H, et al. A dirt-t approach to unsupervised domain adaptation [J]. arXiv preprint arXiv:1802.08735, 2018.

[6] SAITO K, WATANABE K, USHIKU Y, et al. Maximum classifier discrepancy for unsupervised domain adaptation[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2018: 3723-3732.

[7] YOU K, LONG M, CAO Z, et al. Universal domain adaptation[C]//Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2019: 2720-2729.

[8] SUN Y, WANG X, LIU Z, et al. Test-time training with self-supervision for generalization under distribution shifts[C]//International conference on machine learning. PMLR, 2020: 9229-9248.

[9] LI H, PAN S J, WANG S, et al. Domain generalization with adversarial feature learning [C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2018: 5400-5409.

[10] QIAO F, ZHAO L, PENG X. Learning to learn single domain generalization[C]// Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2020: 12556-12565.